Momento 2

Crash Bandicoot en la misión rescatar el Río Medellín

Proyecto final

Momento histórico: Saneamiento del río Medellín, plantas San Fernando y Aguas Claras

Versión para revisión de alcance

1. Contexto y objetivo

La idea es inspirada en el río que vemos todos los días los que usamos el metro como medio de transporte, siempre al pasar por la estación Ayurá vemos a San Fernando y sabemos por historia que la otra Aguas Claras en la autopista norte luego del municipio de Bello, la idea es animar a Crash Bandicoot como operario para ayudar a la recuperación del Rio, no pretendo reproducir las 4 grandes etapas que tienen las plantas pero si generar conciencia de una manera lúdica de la importancia de limpiar nuestro río.

2. Nivel 1

Crash debe recoger las piezas faltantes para poner en marcha la planta de tratamiento de San Fernando sorteando obstáculos para llevarlas a la máquina central.

Vista: plataformas laterales con desplazamiento en el eje x y cámara con scroll limitado.

Controles: Izquierda con la tecla A y derecha tecla D para desplazar, saltar con espacio y x para girar.

Interacción: Recoger componentes con la tecla C, saltar entre superficies, girar para retirar obstáculos.

Retos: Huecos, barreras oscilantes, objetos que caen y superficies temporales.

Objetivo: Reunir componentes y llegar a la máquina conservando los puntos de vida.

Pierde: Cuando la energía llegue a cero o se caiga por un hueco.

Físicas: parabólica de salto, oscilación por periodos de barreras y aceleración de cosas que caen.

3. Nivel 2

Crash tiene en contra un cronometro y debe abrir las compuertas de la planta y conducir el flujo a las áreas correctas antes que se agote el tiempo.

Vista: Cenital fija con varias estaciones, compuertas y diferentes rutas.

Controles: WASD para cada moverse en todas las direcciones y C para activar compuertas.

Interacción: elegir compuertas de separación, recoger residuos que bloquean conductos,

Retos: Llevar el agua por la ruta correcta y reaccionar a obstáculos.

Objetivo: completar 3 zonas.

Tiempo: temporizador de 120 segundos.

Pierde: se termine el tiempo sin completar las zonas.

Físicas: Oscilación de mecanismos, aceleración de residuos empujados por la caída por gravedad.

4. Físicas

Movimiento será calculado con funciones propias de C++ a partir de tiempo transcurrido, movimiento rectilíneo uniforme no se presenta como una de las 3 siguientes:

Parabólico/ Gravedad $y(t) = y0+vy0.t + (g.t2)/2$, y0, velocidad inicial y gravedad, arco de salto y objetos que caen.

Oscilación armónica $x(t)= xc + A. sen (w.t.g)$ Centro, amplitud, frecuencia y fase; compuertas y barrera móviles.

Movimiento acelerado $x(t) = x0+v0.t +(a.t2)/2$ Posición, velocidad y aceleración, residuos que reciben impulso del flujo.

Signos, direcciones, límites de colisión se ajustarán al sistema de coordenadas de la pantalla, el modelo físico es una simplificación jugable, no una simulación hidráulica real.

5. Personalidad y características

Giro, Crash como en el juego de play station puede despejar objetos usando su singular giro durante una breve animación, tiene distancia, alcance y un intervalo de reset, no es permanente. Salto ágil supera vacíos y plataformas el alcance depende de la velocidad inicial y la gravedad. Ambos cumplen con las reglas de cada nivel y no son solo decoración.

6. Agente autónomo, un robot que recolecta

Un robot pequeño como los que presentó Jesen Huang en Nvidia esta semana, tiene como objetivo interceptar residuos de manera autónoma, sin que el jugar indique cada movimiento, con cuatro componentes:

Percepción: detectar residuos y trayectoria, registre posiciones, velocidad y cercanía a una estación crítica.

Razonamiento:: Asignar prioridad según riesgo, distancia y llegada, elegir ruta viable.

Acción: Se desplaza hacia el objetico y ejecuta interacción de recolección.

Aprendizaje: guarda resultados de rutas y actualiza puntajes de éxito, el historial favorece mejores rutas.

Lee estado de residuos y estaciones, sino hay residuos patrulla, calcula prioridad y objetivo, elige rutas aprendidas, ejecutas desplazamientos y recolecta, registra éxito o fallo y actualiza siguiente, repite el proceso. Con un arreglo de puntajes y otro de intentos, el aprendizaje se valida comparando elecciones antes y después de varios intentos.

7. Diseño lógico

Se propone diagrama de clases, se usa herencia: NivelRio y NivelPTAR derivadas de nivel, compartes interfaz de actualización y reglas generales pero con objetivo y comportamiento particulares.

Hay relaciones y estructuras manteniéndose activo, cada nivel contiene personaje y colección de obstáculos, la PTAR contiene agente recolector, las colecciones y puntajes con su justificada implementación, la gestión de instancia seguirá diseño y lo necesario para implementación.

UML Anexo aparte.

8. Inventario

Se anexaran los sprites por nivel los cuales serán en el nivel 1, crash en reposo, salto, obstáculo y superficie. En nivel 2, operar compuerta, mecanismo de control, residuo móvil, agente (robot)